

A Practical Course on Mathematical Modelling

Lecture 3 - Linear Programming models and Graph algorithms

Prof. Viet Hung Nguyen

Clermont Auvergne University - Clermont-Ferrand - France

THREE PARTS, ONE JULIA TOOLBOX

Part I — Primal, dual and sensitivity

A two-variable LP: primal–dual pair, geometry, shadow prices, RHS and objective sensitivity.

Part II — Progressive LP modelling with JuMP

Demand → stocks → working time → load balancing → fractional objectives → scenario MILP.

Part III — Graph algorithms in Julia

Graphs.jl → DataStructures.jl → Dijkstra → one / all shortest paths.

A Two-Variable Primal–Dual Pair

PART I · PRIMAL AND DUAL

Primal (P)

$$\begin{array}{ll}\max & 3x_1 + 2x_2 \\ \text{s.t.} & x_1 + x_2 \leq 4 \\ & 2x_1 + x_2 \leq 6 \\ & x_1, x_2 \geq 0\end{array}$$

Dual (D)

$$\begin{array}{ll}\min & 4y_1 + 6y_2 \\ \text{s.t.} & y_1 + 2y_2 \geq 3 \\ & y_1 + y_2 \geq 2 \\ & y_1, y_2 \geq 0\end{array}$$

Solutions

$$x^* = (2, 2), \quad y^* = (1, 1), \quad z_P^* = z_D^* = 10.$$

Strong duality: the two optimal values coincide.

Solving the Primal with JuMP

PART I · THE PRIMAL MODEL

```
function solve_primal(b1=4.0, b2=6.0)
    model = Model(HiGHS.Optimizer)
    set_silent(model)
    @variable(model, x[1:2] >= 0)
    @objective(model, Max, 3x[1] + 2x[2])
    c1 = @constraint(model, x[1] + x[2] <= b1)
    c2 = @constraint(model, 2x[1] + x[2] <= b2)
    optimize!(model)
    return value.(x), objective_value(model),
           [shadow_price(c1), shadow_price(c2)]
end
```

Keep the constraint reference.

`c1 = @constraint(...)` stores a handle, so that we can query it after solving.

`shadow_price(c)`

Marginal value of the right-hand side of `c`: the corresponding dual variable.

Parametrized RHS.

b_1, b_2 are arguments, which makes the sensitivity study a simple loop.

Solving the Dual with JuMP

PART I · THE DUAL MODEL

```
function solve_dual(c1=3.0, c2=2.0)
    model = Model(HiGHS.Optimizer)
    set_silent(model)
    @variable(model, y[1:2] >= 0)
    @objective(model, Min, 4y[1] + 6y[2])
    @constraint(model, y[1] + 2y[2] >= c1)
    @constraint(model, y[1] + y[2] >= c2)
    optimize!(model)
    return value.(y), objective_value(model)
end
```

Same data, transposed.

The primal RHS (4,6) becomes the dual objective; the primal costs (3,2) become the dual RHS.

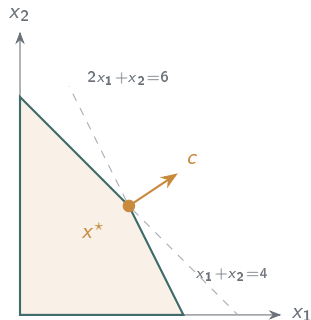
Check.

`solve_dual()` returns $y^* = (1, 1)$ and objective 10: identical to the primal optimum, and equal to the primal shadow prices.

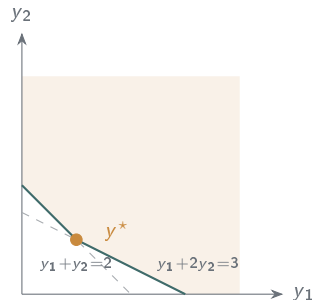
Geometry of the Primal and the Dual

PART I · SEEING THE OPTIMA

Primal in (x_1, x_2)



Dual in (y_1, y_2)



The primal optimum $x^* = (2, 2)$ sits at the intersection of both constraints. The dual region is unbounded: only the window $[0, 4]^2$ is drawn, and its optimum is $y^* = (1, 1)$.

PART I · MARGINAL VALUE OF A RESOURCE

Dual variables = marginal values

While the optimal basis remains unchanged,

$$y_1^* = \frac{\partial z^*}{\partial b_1}, \quad y_2^* = \frac{\partial z^*}{\partial b_2}.$$

Relaxing constraint i by one unit raises the optimal profit by y_i^* .

If both resource constraints stay active,

$$x_1 + x_2 = b_1, \quad 2x_1 + x_2 = b_2 \quad \implies \quad x_1 = b_2 - b_1, \quad x_2 = 2b_1 - b_2.$$

Subtracting the two equations gives x_1 ; substituting back gives x_2 . The optimal value follows:

$$z = 3x_1 + 2x_2 = b_1 + b_2.$$

PART I · HOW FAR CAN WE PUSH b_1 AND b_2 ?

The same basis remains optimal as long as $x_1 = b_2 - b_1 \geq 0$ and $x_2 = 2b_1 - b_2 \geq 0$, that is

$$\frac{b_2}{2} \leq b_1 \leq b_2.$$

Variation	Same basis feasible	Optimal value
b_1 with $b_2 = 6$	$3 \leq b_1 \leq 6$	$z^* = 6 + b_1$
b_2 with $b_1 = 4$	$4 \leq b_2 \leq 8$	$z^* = 4 + b_2$

Both marginal values equal 1 in these intervals, in agreement with $y^* = (1, 1)$. Outside them the basis changes and the shadow price changes too.

Objective-Coefficient Sensitivity

PART I · HOW LONG DOES THE VERTEX STAY OPTIMAL?

At $x^* = (2, 2)$ the two active constraints have slopes -1 and -2 . The vertex stays optimal as long as the objective line, of slope $-c_1/c_2$, lies between them:

$$1 \leq \frac{c_1}{c_2} \leq 2.$$

Keeping $c_2 = 2$

$$2 \leq c_1 \leq 4$$

Keeping $c_1 = 3$

$$\frac{3}{2} \leq c_2 \leq 3$$

At an endpoint, the objective line is parallel to one active constraint: alternative optimal solutions occur along an edge.

PART II · ONE FACTORY, THREE PRODUCTS

The production example has three products.

Product	Unit profit	Max demand
Standard	38	80
Premium	62	55
Eco	50	70

The basic JuMP workflow

```
model = Model(HiGHS.Optimizer)
@variable(model, ...)
@constraint(model, ...)
@objective(model, Max, ...)
optimize!(model)
```

The macros mirror the mathematical notation: variables, constraints, objective, then solve.

Step 1 — Demand Only

PART II · THE SMALLEST MEANINGFUL MODEL

$$\begin{array}{ll}\max & \sum_{p \in P} c_p q_p \\ \text{s.t.} & 0 \leq q_p \leq d_p, \quad p \in P\end{array}$$

All profits are positive, so the optimum simply produces each product up to its demand.

In JuMP

```
@variable(model, q[p in P] >= 0)

@constraint(model, demand[p in P],
             q[p] <= data.demand[p])

@objective(model, Max,
            sum(data.profit[p] * q[p] for p in P))
```

New idea: one indexed variable and one indexed constraint family, created in a single line each.

Step 2 — Stock Constraints

PART II · THREE PHYSICAL RESOURCES

Each unit of product p consumes a_{rp} units of resource r .

For each resource $r \in R$:

$$\sum_{p \in P} a_{rp} q_p \leq S_r$$

Resource	Standard	Premium	Eco	Stock
Steel	2.0	3.2	1.6	260
Electronics	0.8	1.6	1.1	140
Packaging	1.0	1.4	1.8	230

A constraint no longer describes one product alone: it bounds the **aggregate** consumption generated by all products. Convention: `resource_use[x,p]`, rows = resources, columns = products.

Step 3 — Working-Time Constraints

PART II · THREE DEPARTMENTS

Each unit of product p needs h_{dp} hours in department d .

Department	Std	Prem	Eco	Hours
Assembly	1.1	1.9	1.4	165
Finishing	0.8	1.3	1.5	130
Quality control	0.5	0.9	0.8	95

For each department $d \in D$:

$$\sum_{p \in P} h_{dp} q_p \leq H_d$$

Same structure as the stock constraints, with another meaning: many linear constraints look alike even when they model different physical limits.

Step 3 — The Complete JuMP Model

PART II · DEMAND + STOCK + WORKING TIME

```
P = eachindex(data.product_names)
R = eachindex(data.resource_names)
D = eachindex(data.department_names)

model = Model(HiGHS.Optimizer)
@variable(model, q[p in P] >= 0)
@constraint(model, demand[p in P], q[p] <= data.demand[p])
@constraint(model, stock[r in R],
    sum(data.resource_use[r,p] * q[p] for p in P) <= data.stock[r])
@constraint(model, hours[d in D],
    sum(data.hours_use[d,p] * q[p] for p in P) <= data.hours_available[d])
@objective(model, Max, sum(data.profit[p] * q[p] for p in P))
optimize!(model)
```

A Recurring Modelling Pattern

PART II · INDEX SET + COEFFICIENT MATRIX

An index set + a coefficient matrix $\Rightarrow$ a whole family of linear constraints

Family	Index	Coefficients	Right-hand side
demand	$p \in P$	1 (on q_p)	d_p
stock	$r \in R$	a_{rp}	S_r
hours	$d \in D$	h_{dp}	H_d

The same JuMP syntax models stock, working time and many other aggregate resource limits.

Step 4 — Load Balancing Between Teams

PART II · BALANCE UTILIZATION, NOT HOURS

Three teams have **different capacities**, so equalizing absolute hours would be unfair. Balance the utilization rates instead:

$$L_t = \sum_{p \in P} b_{tp} q_p, \quad u_t = \frac{L_t}{C_t}.$$

A team with capacity 150 and a team with capacity 120 should not receive the same number of hours.

Team	Std	Prem	Eco	Cap.
A	1.2	1.6	1.0	150
B	0.7	1.2	1.7	140
C	0.9	0.8	1.3	120

Step 4 — Minimize the Maximum Deviation

PART II · LINEARIZING AN ABSOLUTE VALUE

Let $\bar{u}$ be the average utilization and Δ the largest deviation from it:

$$\min \Delta \quad \text{s.t.} \quad L_t = C_t u_t, \quad u_t - \bar{u} \leq \Delta, \quad \bar{u} - u_t \leq \Delta.$$

$|u_t - \bar{u}| \leq \Delta$ is replaced by two linear inequalities.

Profit floor. To avoid sacrificing too much profit,

$$\sum_{p \in P} c_p q_p \geq 0.95 Z^*,$$

where Z^* is the optimal profit of Step 3 — a previous optimal value reused as a parameter.

Step 4 — Essential JuMP Additions

PART II · EXPRESSIONS AND AUXILIARY VARIABLES

```
@variable(model, util[t in T] >= 0)
@variable(model, average_util >= 0)
@variable(model, delta >= 0)

@expression(model, team_load[t in T],
    sum(data.team_hours[t,p] * q[p] for p in P))

@constraint(model, utilization[t in T],
    team_load[t] == data.team_capacity[t] * util[t])
@constraint(model, avg,
    average_util == sum(util[t] for t in T) / length(T))
@constraint(model, upper[t in T], util[t] - average_util <= delta)
@constraint(model, lower[t in T], average_util - util[t] <= delta)
@constraint(model, profit_floor,
    sum(data.profit[p] * q[p] for p in P) >= 0.95 * Zstar)
@objective(model, Min, delta)
```

Step 5 — Maximize Profit per Labour Hour

PART II · A LINEAR-FRACTIONAL OBJECTIVE

With total profit $P(q) = \sum_p c_p q_p$ and total labour $L(q) = \sum_p \ell_p q_p$, the firm wants

$$\max \frac{P(q)}{L(q)} \quad \text{— not linear.}$$

Charnes–Cooper transformation

Set $\tau = \frac{1}{L(q)}$ and $z_p = \tau q_p$. Then

$$\sum_p \ell_p z_p = 1, \quad Az \leq b \tau, \quad \max \sum_p c_p z_p.$$

A ratio is invariant under scaling, so the transformed model identifies an optimal production *direction*; the largest feasible scale is computed afterwards.

Step 5 — Essential JuMP Pattern

PART II · SCALED VARIABLES z AND τ

```
@variable(model, z[p in P] >= 0)
@variable(model, tau >= 0)

@constraint(model, normalization,
    sum(ell[p] * z[p] for p in P) == 1)
@constraint(model, demand[p in P], z[p] <= data.demand[p] * tau)
@constraint(model, stock[r in R],
    sum(data.resource_use[r,p] * z[p] for p in P) <= data.stock[r] * tau)
@constraint(model, hours[d in D],
    sum(data.hours_use[d,p] * z[p] for p in P) <= data.hours_available[d] * tau)
@objective(model, Max, sum(data.profit[p] * z[p] for p in P))
```

The normalization replaces the denominator of the ratio. Each right-hand side is multiplied by τ . After optimizing, the largest feasible scale is computed to display concrete quantities.

Step 6 — Minimize Material per Unit of Profit

PART II · THE SAME TRICK, ANOTHER DENOMINATOR

With total material $M(q) = \sum_p m_p q_p$ and profit $P(q) = \sum_p c_p q_p$,

$$\min \frac{M(q)}{P(q)}.$$

Now the denominator is profit: use $\tau = \frac{1}{P(q)}$ and $z_p = \tau q_p$.

$$\sum_p c_p z_p = 1, \quad \min \sum_p m_p z_p.$$

The scaled feasibility constraints are identical to Step 5. This shows how one transformation is reused for different linear-fractional objectives. It requires strictly positive total profit, which holds here.

Step 7 — Probabilistic Stock Reserve

PART II · STEEL IS A CRITICAL RESOURCE

The model requires two levels of protection:

- 1 hard steel stock feasibility in **every** scenario;
- 2 steel usage at most 90% of stock in at least 95% of the scenarios.

Chance constraint

$$\Pr(\text{steel consumption} \leq 0.90 \times \text{steel stock}) \geq 0.95$$

Uncertainty comes from small variations in the actual steel coefficients.

Step 7 — Scenario Approximation (MILP)

PART II · BINARY VIOLATION VARIABLES AND BIG-M

Scenario s multiplies the steel coefficients by γ_s .

Hard stock:
$$\sum_p \gamma_s a_{rp} q_p \leq S_r \quad s \in S$$

Reserve:
$$\sum_p \gamma_s a_{rp} q_p \leq 0.90 S_r + M_s y_s, \quad y_s \in \{0, 1\} \quad s \in S$$

Chance level:
$$\sum_{s \in S} y_s \leq 1$$

$y_s = 0$: reserve respected in scenario s ; $y_s = 1$: it may be violated. With 20 equally likely scenarios, a 95% requirement allows at most one violation.

Normalization. With $w_p = q_p / S_r$ the reserve becomes $\sum_p \gamma_s a_{rp} w_p \leq 0.90 + \tilde{M}_s y_s$. The denominator is a constant, so this is a normalization *analogous* to Charnes–Cooper, not a full linear-fractional transformation.

Step 7 — Core JuMP Structure

PART II · BINARY VARIABLES MAKE IT A MILP

```
@variable(model, q[p in P] >= 0)
@variable(model, w[p in P] >= 0)
@variable(model, reserve_violation[s in S], Bin)

@constraint(model, scaled[p in P], q[p] == stockcrit * w[p])
@constraint(model, hard[s in S],
    sum(gamma[s] * a[p] * q[p] for p in P) <= stockcrit)
@constraint(model, reserve[s in S],
    sum(gamma[s] * a[p] * w[p] for p in P)
    <= 0.90 + big_m[s] * reserve_violation[s])
@constraint(model, chance,
    sum(reserve_violation[s] for s in S) <= allowed_violations)
```


Matrix versus Dictionary Data

PART II · TWO WAYS TO REPRESENT THE DATA

Matrices

```
resource_use[r,p]  
hours_use[d,p]  
team_hours[t,p]
```

Compact, efficient, close to mathematical notation.

For introductory LP modelling, the matrix representation is usually the clearer starting point once the meaning of rows and columns has been stated.

Dictionaries

```
resource_use_dict[("Steel", "Premium")]
```

More explicit; handy for irregular sets or externally imported data.

Progressive Sequence at a Glance

PART II · ONE NEW IDEA AT EACH STEP

Step	New modelling concept	New JuMP concept
1	production and demand	variables, indexed constraints, objective
2	stock limits	matrix coefficients, resource-indexed constraints
3	working-time limits	another indexed constraint family
4	load balancing	expressions, auxiliary variables, absolute-value linearization
5	profit per labour hour	Charnes–Cooper for a maximization ratio
6	material per profit	Charnes–Cooper for a minimization ratio
7	probabilistic reserve	scenario MILP, binaries, normalized stock usage

PART III · VERTICES, ARCS, NEIGHBORHOODS

```
using Graphs
```

```
G = SimpleDiGraph(5)
add_edge!(G, 1, 2)
add_edge!(G, 1, 3)
```

```
nv(G)           # number of vertices
ne(G)           # number of arcs
outneighbors(G, 1) # successors of 1
inneighbors(G, 3)  # predecessors of 3
has_edge(G, 1, 2)
```

What Graphs.jl stores

Vertices, edges/arcs, neighborhoods, degrees, and many classical graph algorithms.

The key operation for Dijkstra

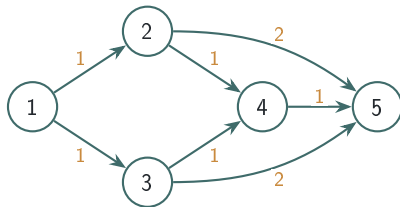
```
for v in outneighbors(G, u)
    # relax arc (u,v)
end
```

A Weighted Directed Graph

PART III · TOPOLOGY IN THE GRAPH, LENGTHS IN A DICTIONARY

```
w = Dict(  
    (1,2) => 1.0, (1,3) => 1.0,  
    (2,4) => 1.0, (3,4) => 1.0,  
    (2,5) => 2.0, (3,5) => 2.0,  
    (4,5) => 1.0,  
)  
  
G = SimpleDiGraph(5)  
for (u,v) in keys(w)  
    add_edge!(G, u, v)  
end
```

$w[(u,v)]$ is the length of arc (u,v) . The example is chosen so that several shortest paths from 1 to 5 exist.



Weights for the Built-in Routines

PART III · THE DISTANCE MATRIX

The built-in shortest-path routines of `Graphs.jl` read the lengths from a matrix `distmx`, where `distmx[u,v]` is the length of arc (u, v) :

```
distmx = fill{Inf, nv(G), nv(G)}
for ((u,v), c) in w
    distmx[u,v] = c
end
```

Our own implementation reads arc lengths from the dictionary `w`; the built-in one reads them from `distmx`. Both describe the same weighted graph.

PART III · WHICH VERTEX HAS THE SMALLEST TENTATIVE DISTANCE?

```
using DataStructures

H = BinaryMinHeap{Tuple{Float64,Int}}{()}
push!(H, (5.0, 3))
push!(H, (2.0, 2))
push!(H, (0.0, 1))

first(H)      # minimum pair
pop!(H)       # remove and return the minimum
isempty(H)
```

Dijkstra repeatedly asks for the unprocessed vertex with the **smallest tentative distance**: a min-heap supports this efficiently.

Julia compares tuples lexicographically, so $(2.0, 4) < (5.0, 2)$. A heap of (distance, vertex) pairs therefore extracts the smallest distance first.

A BinaryMinHeap is not meant to be traversed with `collect(H)`. Read its root with `first(H)` and extract with `pop!(H)`.

Dijkstra's Algorithm

PART III · ONE SHORTEST PATH

For a source vertex s :

- 1 initialize $d_s = 0$ and $d_v = +\infty$ for $v \neq s$;
- 2 extract the vertex u with the smallest tentative distance;
- 3 for each arc (u, v) , **relax** it:

$$\text{newdist} = d_u + w_{uv}, \quad \text{if } \text{newdist} < d_v : d_v \leftarrow \text{newdist}, \text{ parent}(v) \leftarrow u.$$

Requirement: arc lengths must be **nonnegative**.

The predecessor $\text{parent}(v)$ lets us reconstruct one shortest path by walking backward from the target.

Dijkstra in Julia — Initialization

PART III · DISTANCES, PARENTS AND A HEAP

```
function dijkstra_one_path(G, w, source)
    n = nv(G)
    dist = fill(Inf, n)
    parent = fill(0, n)
    dist[source] = 0.0

    H = BinaryMinHeap{Tuple{Float64,Int}}{()}
    push!(H, (0.0, source))

    # ... main loop on the next slide ...
    return dist, parent
end
```

dist

Tentative distances, $+\infty$ everywhere except the source.

parent

One predecessor per vertex; 0 means “none yet”.

H

Min-heap of (distance, vertex) pairs, started with the source.

Dijkstra in Julia — Main Loop and Relaxation

PART III · EXTRACT, IGNORE OBSOLETE ENTRIES, RELAX

```
while !isempty(H)
    du, u = pop!(H)
    du > dist[u] && continue

    for v in outneighbors(G, u)
        newdist = du + w[(u,v)]
        if newdist < dist[v]
            dist[v] = newdist
            parent[v] = u
            push!(H, (newdist, v))
        end
    end
end
```

Extraction

`pop!(H)` returns the pair with the smallest distance.

Relaxation

If the path through u is shorter, update `dist[v]`, record `parent[v]`, and push the new pair.

Lazy Heap Updates

PART III · WHY `du > dist[u] && continue`?

When the distance of a vertex improves, we simply insert a **new pair**:

```
push!(H, (newdist, v))
```

We never search for and modify the old heap entry. Later, an obsolete pair is ignored by:

```
du > dist[u] && continue
```

This makes the implementation much simpler than an explicit *decrease-key* operation: a stale pair has a distance larger than the current `dist[u]`, so it is skipped when extracted.

Reconstructing One Path

PART III · FOLLOW THE PARENTS BACKWARD

```
function reconstruct_path(parent, source, target)
    target == source && return [source]
    parent[target] == 0 && return Int[]

    path = Int[]
    v = target
    while v != 0
        push!(path, v)
        v == source && break
        v = parent[v]
    end
    reverse!(path)
    return path
end
```

Two early exits

The target is the source, or it was never reached (empty path).

Backward walk

Collect vertices from the target back to the source, then reverse the list.

Comparison with the Built-in Dijkstra

PART III · WHAT SHOULD WE COMPARE?

```
state = dijkstra_shortest_paths(G, 1, distmx)
state.dists
state.parents
enumerate_paths(state, 5)
```

`DijkstraState` holds the shortest distances and one predecessor per vertex.

When several shortest paths exist, two correct implementations may return **different** paths, e.g. $1 \rightarrow 2 \rightarrow 5$ versus $1 \rightarrow 3 \rightarrow 5$.

Compare (i) the shortest distances, and (ii) the length of the returned path — not necessarily the vertex sequence.

From One Shortest Path to All Shortest Paths

PART III · TIES ARE LOST WITH A SINGLE PARENT

A single parent loses ties. Replace `parent[v]::Int` by a list `pred[v]::Vector{Int}`.

For an arc (u, v) :

$$\begin{cases} d_u + w_{uv} < d_v : & d_v \leftarrow d_u + w_{uv}, \quad \text{pred}(v) \leftarrow \{u\}, \\ d_u + w_{uv} = d_v : & \text{pred}(v) \leftarrow \text{pred}(v) \cup \{u\}. \end{cases}$$

Strict improvement discards the old predecessors; a tie adds the new one. The lists compactly represent the **shortest-path predecessor graph**.

All Predecessors — What Changes in the Code

PART III · dijkstra_all_predecessors

Initialization

```
dist = fill(Inf, n)
pred = [Int[] for _ in 1:n]
dist[source] = 0.0
```

One empty list per vertex instead of `parent = fill(0,n)`.

Relaxation

```
newdist = du + w[(u,v)]
if newdist < dist[v]
    dist[v] = newdist
    pred[v] = [u]
    push!(H, (newdist, v))
elseif newdist == dist[v]
    push!(pred[v], u)
end
```

A tie now records an extra predecessor.

The rest (heap, lazy updates, return `dist`, `pred`) is identical to `dijkstra_one_path`.

Enumerating All Shortest Paths

PART III · A SEPARATE, RECURSIVE SECOND STEP

```
function enumerate_shortest_paths(pred, source, target)
  target == source && return [[source]]
  isempty(pred[target]) && return Vector{Vector{Int}}()

  paths = Vector{Vector{Int}}()
  for u in pred[target]
    for p in enumerate_shortest_paths(pred, source, u)
      push!(paths, [p; target])
    end
  end
  return paths
end
```

Idea

Every shortest path to target ends with an arc from one of its predecessors u :
recurse on each u and append target.

Two phases

Dijkstra computes distances and predecessor lists; path enumeration is a separate step.

Result on the Example

PART III · FROM 1 TO 5

Shortest distance = 3

Four shortest paths:

- $1 \rightarrow 2 \rightarrow 5$
- $1 \rightarrow 3 \rightarrow 5$
- $1 \rightarrow 2 \rightarrow 4 \rightarrow 5$
- $1 \rightarrow 3 \rightarrow 4 \rightarrow 5$

Built-in all-paths mode

```
state = dijkstra_shortest_paths(  
    G, 1, distmx; allpaths=true)  
state.predecessors  
state.pathcounts  
enumerate_shortest_paths(  
    state.predecessors, 1, 5)
```

`state.predecessors` is directly comparable with our array `pred` (e.g. `pred[4]=[2,3]`), and `pathcounts[5]` gives the number of shortest paths.

PART III · COMPUTING VERSUS LISTING

Dijkstra

With adjacency lists and a binary heap, with $n = |V|$ and $m = |A|$,

$$O((n + m) \log n).$$

Enumeration is another matter. The number of shortest paths may be **exponential**, so explicitly listing all of them cannot in general be polynomial in the input size: the output itself may be huge.

The recursive enumeration is simplest with strictly positive arc lengths. With zero-length cycles the predecessor relation can contain cycles, so cycle protection is needed if only simple paths are desired.

Overall Take-away

PUTTING IT TOGETHER

In Julia, a mathematical algorithm is assembled from complementary components:

- `Graphs.jl` — domain structures: vertices, arcs, neighborhoods;
- `DataStructures.jl` — efficient data structures such as heaps;
- `JuMP` — modelling syntax close to the algebraic formulation;
- `HiGHS` — the numerical LP/MILP solver.

The modelling workflow is equally progressive: formulate the mathematical quantity first, then introduce variables and constraints, and only then translate them into JuMP syntax.